

SprintWell

Planificación de sprints orientada al bienestar del equipo

Trabajo Fin de Máster · Máster de Desarrollo con IA

Las herramientas de hoy optimizan capacidad y deadlines.

Las preferencias del trabajador quedan fuera del modelo.

Este TFM las mete dentro.

El problema

- Jira / Linear / Asana modelan capacidad, dependencias, prioridad, *skill* — **no** las preferencias del trabajador.
- Cuando se atienden, es **ad-hoc**: Slack, hoja de cálculo, sensibilidad del *team lead*.
- Consecuencia: sesgo hacia lo medible e **inequidad silenciosa** (alguien siempre carga con lo que nadie quiere).

La propuesta y 3 hipótesis

SprintWell: preferencias como **objetivo de primer orden** + equidad seleccionable.

- **H1 — modelado:** un DSL de reglas cubre las preferencias típicas de forma tratable.
- **H2 — algorítmica:** CP-SAT resuelve instancias realistas con calidad interactiva.
- **H3 — metodológica:** una persona + IA construye esto en un TFM, **con métricas**.

Contexto y alcance

- 12 semanas · un desarrollador · datos sintéticos.
- Contribución **triple**: **producto** (pesa más) · **algoritmia** · **metodología**.
- Exclusiones deliberadas (multi-tenant, NLP, usuarios reales) — decisiones, no descuidos.

Modelo formal

- Variables: asignación x_{ij} , inicio s_i ; restricciones duras **R1–R7**.
- Felicidad por persona: $f_j = \frac{\sum_r w_r c_r}{\sum_r w_r} \in [0, 1]$
- **Tres modos de equidad:** utilitarista (suma) · max-min (Rawls) · Nash (producto).
- **NP-difícil** (reducción desde GAP) → justifica un solver heurístico-completo.

Arquitectura

Tres servicios independientes:

Servicio	Stack
Backend	NestJS · DDD 4 capas · Prisma · PostgreSQL
Optimizer	Python · FastAPI · OR-Tools CP-SAT
Frontend	React · Vite · Tailwind · cliente tipado desde OpenAPI



DEMO EN VIVO

login → sprint + tareas → editor de reglas
→ **planificar (Nash)** → Gantt + bienestar
→ **comparar Nash vs Utilitario** → explicabilidad
(vídeo de respaldo por si falla el directo)

Resultados — CP-SAT vs baselines

Algoritmo	Felicidad media	Mín	Reglas satisfechas
CP-SAT	0.81	0.50	82 %
<i>greedy</i>	0.49	0.00	55 %
aleatorio	0.44	0.00	51 %

El piso importa: los baselines dejan a alguien en 0.0 ;
CP-SAT no.

Resultados — modos de equidad

Caso **Apollo** (mismo sprint, tres modos):

- **Nash** → mejor equilibrio: media **0.87**, piso **0.60**.
- **Hugo (junior)** pasa de **0.50** → **1.00** al activar la equidad.
- Escalabilidad **honesta**: resuelve hasta ~150 tareas / 20 personas; techo en XL.

Metodología con IA

- **85 PRs** mezclados · **419 commits** atómicos · CI en verde por PR.
- Protocolo **delegar / revisar / rechazar**; cobertura de test por capa.
- Huecos *upstream* detectados y corregidos (instancias infactibles, endpoints faltantes) → **no *vibe coding***.

Conclusiones

- **H1** validada · **H2** validada con matices (rango acotado)
· **H3** validada con reservas.
- Aporte más duradero: hacer **explícitas y explicables** las decisiones de equidad y preferencia, antes ocultas.
- Trabajo futuro: normalizar equidad, persistir explicabilidad, escalar el solver, **estudio con usuarios reales**.

Gracias

**Preferencias dentro del modelo,
decisiones de equidad a la vista.**

github.com/josec-hdez/SprintWell

¿Preguntas?